DeepScientist

AI REVIEW REPORT

AIG_CUR_2026_FARS_draft_and_continue

Source file: AIG_CUR_2026_FARS_draft_and_continue.pdf

Document No.	515a9183-5d04-4d06-8c0a-31c35f48bcf9	Work ID	DS-RV-515A91835
Status	Completed	Agent Model	gpt-5.2
Token Usage	Input 9752199 | Output 118043 | Total 9870242	LLM Requests	64
Generated At	2026-06-10 09:13:43 UTC	Producer	DeepReviewer 2.0

This document is generated for professional review, archival, and collaborative decision making. All conclusions should be interpreted with domain expertise and final human verification.

Final Agent Report

Structured markdown synthesized by the annotation agent.

Summary

This paper studies **Draft-and-Continue Self-Consistency (DCS)**, a two-stage variant of self-consistency that (i) samples multiple *draft* chain-of-thoughts up to a token limit, (ii) forms a vote histogram over *interim answers*, and (iii) spends additional continuation tokens only on high-vote branches (with a hedge to cover the 2nd-place answer). The goal is to improve the accuracy–cost trade-off by reallocating test-time compute rather than uniformly sampling full solutions.

Empirically, on **MATH-500 with Qwen2.5-Math-7B-Instruct**, DCS achieves the same reported accuracy as the strongest baselines (76.8%) but **uses more tokens** than uniform self-consistency and is **Pareto-dominated** by confidence-guided early stopping. The paper's core diagnosis is that the continuation mechanism is largely "switched off" because **~96% of drafts already terminate with a parseable answer under T_{draft}=1024**, so continuation rarely changes outcomes (Page 4 - Experiments: Analysis).

As a negative-result note, the topic is valuable and the manuscript is refreshingly direct about failure modes. However, the current draft has several *publication-critical* gaps that reduce decision confidence in the negative conclusion and its generality: (1) the interim-answer parser and "draft complete" definition are not specified (Page 2 - Method: Stage 1 Draft Sampling), (2) comparisons are not budget-matched across methods (Page 3 - Experiments: Baselines), and (3) the key boundary-condition experiment (a T_{draft} sweep / continuation-activation analysis) is missing despite being the paper's main explanation (Page 3 - Hyperparameters; Page 4 - Analysis). With these fixes, the paper can become a strong, defensible negative result with clear takeaways about when continuation-based branch budgeting is (un)likely to help.

Strengths

1. **Clear problem and honest negative result (good scientific hygiene).** The paper is explicit that DCS is not Pareto-competitive under the evaluated setting and offers a concrete diagnosis (high draft completion makes continuation mostly irrelevant) (Page 4 - Experiments: Analysis; Page 5 - Conclusion).
2. **Simple, understandable mechanism.** DCS is easy to describe and implement in principle (two-stage draft → interim-answer vote → continue selected branches with a hedge). This makes the negative result interpretable and useful for practitioners who might otherwise try similar ideas (Page 2 - Method: Draft-and-Continue).
3. **Relevant baseline family.** The manuscript compares against (i) greedy CoT, (ii) uniform self-consistency, and (iii) a confidence-guided early stopping method—i.e., the most direct "efficiency" competitors in the self-consistency decoding space (Page 3 - Experiments: Baselines).
4. **Cost reporting beyond accuracy.** Reporting average generated tokens and average calls (not only accuracy) is aligned with the test-time compute literature's budget-aware emphasis and is necessary for a credible efficiency discussion (Page 4 - Experiments: Evaluation Metrics).

5. **Actionable qualitative takeaway (even before fixes).** The paper's main insight—that continuation-based branch budgeting only has a chance when the draft budget is materially shorter than typical solution length—is a useful rule-of-thumb for future TTC method design, and aligns with broader budget-aware evaluation perspectives in the literature [8].
6. **Good revision potential.** The core weaknesses (parser definition, matched-budget evaluation, and one key sensitivity sweep) are all fixable with relatively modest additional experimentation and clearer method specification.

Weaknesses

1. Reproducibility gap: interim-answer extraction and "draft complete" are undefined. DCS's core decision signal is the interim-answer vote histogram, yet the manuscript does not specify the deterministic parser/normalization rules or how NULL drafts are handled beyond a brief description (Page 2 - Method: Stage 1 Draft Sampling). This also makes the reported ~96% draft-completion rate difficult to interpret (what counts as "parseable final answer"?).
2. **Unfair / confounded efficiency comparison due to unmatched budgets.** DCS uses ~10 calls (7 drafts + 3 continuations), while Uniform SC uses 5 calls and Greedy uses 1; CGES is adaptive up to 5 (Page 3 - Experiments: Baselines). Without matched-call or matched-token-cap baselines, efficiency claims can reflect budget mismatch rather than algorithmic merit.
3. **Key mechanism boundary is asserted but not experimentally validated.** The paper's main explanation is "drafts are too long," implying DCS might work for smaller T_{draft} or harder/longer tasks (Page 4 - Analysis; Page 5 - Discussion). However, there is no T_{draft} sweep, no continuation-activation analysis, and no length-stratified/harder benchmark test. This leaves the main takeaway as a plausible hypothesis rather than a closed empirical argument.
4. **Token-budget accounting is internally confusing.** The method states a budget upper bound $B\ T_{draft}+k\ T_{cont}$ (Page 3 - Method: Final Aggregation), while Table 1 reports a much smaller realized token count (~4,114). The paper should explicitly present expected realized cost and clarify how continuation behaves when drafts terminate early.
5. **Method randomness is not fully controlled.** "Branches within each answer cluster are selected arbitrarily" (Page 2 - Method: Stage 2). This can inject extra variance and may contribute to DCS's higher observed variance; the selection policy should be deterministic or ablated.
6. **Over-strong generalization / gap statements.** The introduction's framing that early-stopping methods "fundamentally discard partial reasoning" is overstated given related work on prefix-guided generation and pruning (Page 1 - Introduction). Similarly, "Pareto-dominates" and "dominates two-stage approaches" should be bounded to the tested regime unless frontier curves are provided.
7. **Minor but correctness-affecting presentation issues.** Hyperparameter notation contains visible typos/artifacts (e.g., $T_{cont}=i0\ 2\ 4$) (Page 3 - Hyperparameters). In a short paper, these details disproportionately affect reviewer trust.

Key Issues

Ranked Error Board (Top-5)

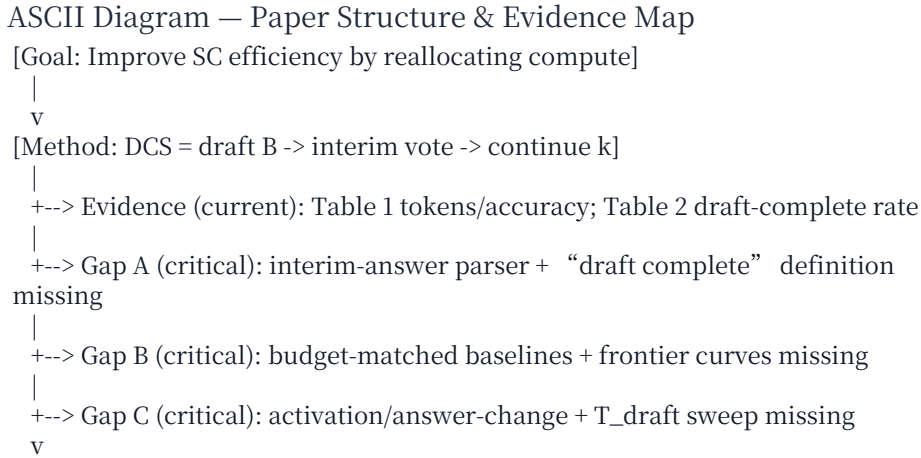
(Severity | Research-Value Impact | Validity Risk | Fixability | Confidence)

1. Undefined interim-answer parser + "draft complete" criterion (Major | High | High | High | High)

- **Anchor:** Page 2 - Method: Stage 1 Draft Sampling.
- **Why it matters:** DCS’s selection signal and the core negative-result diagnosis (~96% completion) depend on parsing; without a precise definition, the main conclusion is not reproducible.
- **Acceptance criterion:** Provide deterministic extraction + normalization + examples; report NULL rate and sensitivity.

2. **Budget-mismatch confounds efficiency claims** (Major | High | High | Medium | High)
   - **Anchor:** Page 3 - Experiments: Baselines; Page 4 - Main Results.
   - **Why it matters:** DCS uses ~10 calls vs `1/5/≤5` for baselines; “efficiency” deltas can be an artifact.
   - **Acceptance criterion:** Add matched-calls and matched-token-cap comparisons; plot accuracy–cost frontiers.
3. **Missing mechanism-boundary validation (T_{draft} sweep / activation / answer-change rates)** (Major | High | Medium | Medium | High)
   - **Anchor:** Page 3 - Hyperparameters; Page 4 - Analysis; Page 5 - Discussion.
   - **Why it matters:** The paper’s explanation is plausible but currently not a closed experimental argument.
   - **Acceptance criterion:** Report continuation activation %, answer-change %, and a T_{draft} sweep under fixed caps.
4. Token accounting and “budget” definition are inconsistent/unclear (Major | Medium | Medium | High | Medium)
   - **Anchor:** Page 3 - Method: token budget formula; Page 4 - Metrics.
   - **Why it matters:** The paper is about efficiency; readers need explicit definitions (generated tokens? include prompts? early termination?).
   - **Acceptance criterion:** Define cost precisely; report expected realized cost; reconcile with Table 1.
5. Method introduces extra randomness (‘arbitrary’ intra-cluster branch selection) (Major | Medium | Medium | High | Medium)
   - **Anchor:** Page 2 - Method: Stage 2.
   - **Why it matters:** May explain higher variance and reduces interpretability.
   - **Acceptance criterion:** Use deterministic tie-breakers; include an ablation for random vs deterministic selection.

ASCII Diagram — Paper Structure & Evidence Map



```
[Conclusion claim: DCS dominated when drafts already finish]
  |
  v
[Needed to close loop: define parser; match budgets; validate boundary]
```

ASCII Diagram — Revision Strategy Roadmap

```
P0 (must, 1–2 days):
 - Define parser + cost accounting
 - Add matched-budget baselines
   -> makes negative result reproducible + fair

P1 (high value, 2–4 days):
 - T_draft sweep + activation/answer-change analysis
   -> turns diagnosis into validated mechanism boundary

P2 (optional, 3–7 days):
 - 1 extra model or harder/longer benchmark
   -> strengthens generality claims
```

Actionable Suggestions

Must (publication-critical)

1. **Specify interim-answer extraction + normalization + NULL handling (reproducibility package).**
   - **Where:** Page 2 - Method: Stage 1 Draft Sampling.
   - **Do:** Add an appendix with deterministic extraction rules (regex / priority order), normalization rules (fractions/decimals, whitespace, sign, boxed vs final line), and examples of partial traces.
   - **Acceptance criteria:** Given any raw completion, a reader can reproduce a_i^{draft} and whether it is NULL.
2. Define "draft complete" and continuation behavior in the dominant regime.
   - **Where:** Page 3 - Method: Final Aggregation; Page 4 - Analysis.
   - **Do:** State whether completed drafts are continued (and how) vs treated as no-op; report continuation activation rate (non-zero continuation tokens).
   - **Acceptance criteria:** Provide a table: activation %, mean continuation tokens conditional on activation, answer-change % conditional on activation.
3. **Add matched-budget evaluations (calls and tokens).**
   - **Where:** Page 3 - Experiments: Baselines; Page 4 - Main Results.
   - **Do:**
     - Matched-calls: Uniform SC with `n=10` vs DCS (`B=7,k=3`).
     - Matched-token-cap: run all methods under the same expected generated-token cap.
   - **Acceptance criteria:** One accuracy–cost plot with curves (vary `n` for SC, γ for CGES, T_{draft}/k for DCS).
4. **Fix token accounting and remove ambiguous budget statements.**
   - **Where:** Page 3 - Method/Hyperparameters.
   - **Do:** Replace $B\ T_{draft}+k\ T_{cont}$ with a clear "worst-case cap" plus an explicit expected realized cost expression; define whether tokens include prompts.
   - **Acceptance criteria:** Table 1's Avg Tokens can be derived from stated accounting choices.

Nice-to-have (high value, but not strictly required)

5. T_{draft} **sensitivity sweep to validate the proposed boundary condition.**

- **Where:** Page 3 - Hyperparameters; Page 5 - Discussion.
- **Do:** Sweep $T_{draft} \in \{128, 256, 512, 1024\}$ and keep total cap comparable (adjust T_{cont} or B, k). Report completion/activation and accuracy–tokens.

6. **Control method randomness and explain variance.**

- **Where:** Page 2 - Stage 2.
- **Do:** Replace "arbitrary" intra-cluster branch selection with a deterministic rule (sampling order). Add an ablation comparing deterministic vs random tie-breaking.

7. Strengthen related-work positioning around "partial-trace reuse".

- **Where:** Page 1 - Introduction gap; Page 5 - Related Work.
- **Do:** Add a short paragraph contrasting continuation-based budgeting (DCS) vs prefix-guided generation [5] and pruning-based SC [4], and cite the closest works explicitly.

Copy-ready paragraph fix (recommended insertion)

Insert after Page 2 - Method: Stage 1 Draft Sampling

Mentor Revised Version:

DCS relies on a deterministic interim-answer parser to construct the vote histogram. We extract a_i^{draft} by (i) selecting the final occurrence of a `"boxed·"` answer when present, (ii) otherwise extracting the last numeric expression in the final line, and (iii) normalizing equivalent formats (e.g., fractions vs decimals, whitespace, and sign). If no parseable answer exists, we assign the draft to a NULL bucket and exclude it from continuation selection. We define a draft as "complete" if it emits an explicit final-answer marker before reaching T_{draft}; otherwise it is "truncated" and eligible for continuation.

ASCII Diagram — Experiment Upgrade Plan

```
P0: Reproducibility + fairness
  - Parser + cost accounting
  - Matched-budget baselines

P1: Validate mechanism boundary
  - T_draft sweep (128..1024)
  - activation % + answer-change %

P2: Generality check
  - 1 harder/longer benchmark or 1 extra model
```

Storyline Options + Writing Outlines

Option A — "Negative-result as boundary condition" (recommended)

- Arc: Problem (SC is expensive) → Proposed continuation idea (DCS) → Negative result under a common regime → Mechanistic diagnosis (drafts already finish) → Boundary condition for when continuation could help.
- Why better: matches the paper's true value (a falsifiable boundary condition) and keeps claims scoped to evidence.

Option B — "Budget-aware evaluation lens"

- Arc: Problem (TTC papers often compare under mismatched compute) → Define a fair budget metric → Evaluate DCS as a case study → Find that early stopping dominates → Provide guidance for TTC evaluation.
- Why better: leverages [8] to frame the contribution as "how to evaluate" plus a concrete negative case.

Option C — “Method-first (engineering note)”

- Arc: Define DCS algorithm precisely → Provide reproducibility package → Benchmark → Analyze failure.
- Why better: if targeting a systems/practice audience; less compelling for research framing.

Abstract Outline (complete; S1–S5)
S1 (Problem & setting): State that self-consistency improves CoT reasoning but is expensive because it samples multiple full traces.

- Evidence anchor: Page 1 - Abstract (problem statement).

S2 (Gap): State that existing efficiency methods mostly stop/prune sampling; pose the complementary question of reallocating tokens to promising partial traces.

- Evidence anchor: Page 1 - Introduction (early-stopping/pruning motivation).

S3 (Method): Define DCS as two-stage draft sampling (B, T_{draft}) → interim-answer histogram → continue k selected branches (T_{cont}) with hedge allocation.

- Evidence anchor: Page 2 - Method (Stages 1–2).

S4 (Main result): Report the key accuracy and cost outcome on MATH-500 + Qwen2.5-Math-7B-Instruct; state that CGES-LNS yields the same accuracy with fewer tokens.

- Evidence anchor: Page 4 - Main Results (Table 1).

S5 (Diagnosis & bounded implication): Report that ~96% drafts complete within $T_{draft}=1024$, so continuation rarely activates; conclude that continuation-based budgeting is only promising when drafts are short relative to solution length.

- Evidence anchor: Page 4 - Analysis (Table 2) and Page 5 - Discussion.

Introduction Outline (complete; paragraph-by-paragraph)
P1 — Stakes + scope + metric definition.

- Role: Establish why test-time compute matters, then *narrow scope* to self-consistency-style sampling.
- Target claim: “We study budget-aware self-consistency under constraints measured in generated tokens and model calls.”
- Transition: From general TTC to the specific SC family.
- Required evidence: later sections’ reported metrics (Page 4 - Evaluation Metrics).

P2 — Prior work categories + precise gap.

- Role: Categorize efficiency approaches (early stopping; pruning; prefix/partial-trace reuse), and state what is missing.
- Target claim: “Prior methods reduce cost by stopping or pruning; fewer methods explicitly allocate extra continuation tokens to selected partial traces based on interim answers.”
- Transition: From “what exists” to “what we test.”
- Required evidence: citations to closest families (use only retrieved refs in final paper revision), e.g., early stopping [1][2], pruning [4][6], prefix-guided [5].

P3 — DCS proposal + negative result preview + contribution bullets.

- Role: Describe DCS at high level, preview the negative result, and state the *mechanistic hypothesis* (draft-completion collapse) as something you validate.
- Target claim: "DCS is a probe of continuation-based budgeting; under $T_{draft}=1024$ it is dominated; the key reason is high draft completion."
- Transition: From question to method to what the paper will show.
- Required evidence: Table 1 + Table 2, and (after revision) a T_{draft} sweep + activation analysis.

One concrete title tweak (optional but helpful)
Current title is informative but long and slightly opaque ("branch budget-ing"). A clearer title pattern is *Setting + idea + outcome*:

- Suggested: "Draft-and-Continue Self-Consistency: When Two-Stage Branch Budgeting Fails for MATH Reasoning"
- Or (more general): "When Continuation Doesn't Help: A Negative Result for Two-Stage Budgeting in Self-Consistency"

Priority Revision Plan

P0 (Must) — Make the negative result reproducible and the comparison fair
Goal: A reader can re-run DCS and agree that (under the tested regime) it is not Pareto-competitive.

1. **Parser + definitions package (same day).**
   - Add: interim-answer extraction rules; normalization; NULL bucket policy; "draft complete" definition; continuation prompting for completed drafts.
   - Output: Appendix + one table of parser stats (NULL %, early-answer %, completion %).
2. **Cost accounting cleanup (same day).**
   - Define: generated tokens vs prompt tokens; per-call caps; what counts toward "Avg Tokens."
   - Replace: budget upper bound sentence with worst-case cap + expected realized cost.
3. Budget-matched baselines (1–2 days).
   - Add: Uniform SC with `n=10`; DCS variant matching `le 5` calls; matched-token-cap setting.
   - Output: one frontier plot (accuracy vs expected tokens) with curves.

P1 (High priority) — Validate the paper's own failure-mode hypothesis
Goal: Turn "drafts are too long" from a plausible story into verified mechanism.

4. T_{draft} sweep under fixed caps (2–4 days).
   - Sweep: $T_{draft} \in \{128, 256, 512, 1024\}$.
   - Control: keep total expected tokens comparable by adjusting T_{cont} and/or B, k.
   - Report: completion %, activation %, answer-change %, accuracy, tokens.
5. Continuation-effect decomposition (1–2 days).
   - Report: conditional accuracy on cases where continuation activates vs not.
   - Report: how often continuation flips the final answer (and whether flips improve correctness).

P2 (Nice-to-have) — Strengthen generality and related-work positioning

Goal: Support bounded general claims (or limit claims appropriately).

6. One extra setting (3–7 days).

· Option: one harder/longer benchmark or a length-stratified MATH subset; or one additional model.

7. **Related-work tightening (same day).**

· Add: explicit comparison to prefix-guided SC and pruning-based SC; soften over-strong "discard partial reasoning" language.

ASCII Diagram — Revision Strategy Roadmap

```
[Reproducibility missing] -> [Add parser + cost definitions] -> [Readers can verify DCS]

[Budget mismatch] -> [Matched calls/tokens baselines + frontier plot] -> [Fair efficiency claim]

[Boundary hypothesis untested] -> [T_draft sweep + activation/flip analysis] -> [Mechanistic negative result]

[Generality unclear] -> [1 extra setting] -> [Bounded but stronger takeaway]
```

Experiment Inventory & Research Experiment Plan

Exp ID	Objective/Hypothesis	Setup (data/split/protocol/baselines)	Metrics	Main Outcome	Claim Supported	Current Limitation
E1	Test whether DCS improves SC efficiency by continuing high-vote branches	MATH-500; Qwen2.5-Math-7B-Instruct; DCS vs Greedy CoT vs Uniform SC (n=5) vs CGES-LNS ($b_{max}=5$, $\gamma=0.95$) (Page 3–4)	Exact-match accuracy; avg generated tokens; avg calls; mean±std over 3 seeds (Page 4)	DCS accuracy 76.8% matches best baselines but uses more tokens than Uniform SC (+19.6%) and CGES-LNS (+109%) (Page 4)	Negative result under this regime	Budgets/calls not matched; no frontier curve; CGES-LNS details not specified
E2	Diagnose why DCS fails (continuation rarely matters)	Same as E1; diagnostic stats for DCS across seeds (Table 2)	Fallback %; Draft Complete %; avg tokens	~96.3% draft-complete; fallback ~3.7% (Page 4)	Hypothesis: drafts already finish under T_{draft}, collapsing continuation	"Draft complete" depends on parser/termination definition; no activation/answer-change analysis

Research-Theme Gap Diagnosis

· **New knowledge:** The paper aims to produce a boundary condition for continuation-based budgeting ("when drafts already finish, continuation is wasted"). This is promising, but currently under-validated because the key boundary-variable (T_{draft}) is not swept.

· **Reproducibility:** The most important missing piece is the interim-answer parser + completion criterion; without it, Table 2 is not independently checkable.

- **Impact on practice:** Practitioners care about "tokens vs calls vs latency." The paper reports tokens/calls but lacks matched-budget fairness and latency/cost modeling.

Proposed Research Experiments (P0/P1/P2)

P0-E1 — Matched-budget baselines (calls + token caps)

- **Target Claim:** "DCS is Pareto-dominated by CGES-LNS (and worse than Uniform SC) under equal budgets."
- **Hypothesis:** Under matched calls/tokens, DCS will remain non-competitive at $T_{draft}=1024$.
- **Minimal Design:** Add Uniform SC with n=10; add a DCS variant with total calls ≤ 5; run a matched-token-cap setting where all methods are constrained to the same expected generated tokens.
- **Controls/Baselines:** Greedy; Uniform SC (n=5,10); CGES-LNS (vary γ); DCS variants.
- **Metrics:** Accuracy; expected tokens; calls; optionally latency proxy.
- **Success Criterion:** Conclusion does not change under fair budgets; frontier plot shows DCS dominated at tested regime.
- **Estimated Cost/Time:** 1–2 days.
- **Expected Paper-Quality Gain:** High (removes confound; strengthens negative result).

P1-E1 — T_{draft} sweep with fixed total caps

- **Target Claim:** "Continuation helps only when drafts are short relative to solution length."
- **Hypothesis:** As T_{draft} decreases, draft completion falls and continuation activation rises; DCS may improve (or still fail), but the boundary becomes measurable.
- **Minimal Design:** Sweep $T_{draft} \in \{128, 256, 512, 1024\}$; adjust T_{cont} or (B, k) to keep expected-token budget comparable.
- **Controls/Baselines:** Uniform SC curve (vary n); CGES curve (vary γ).
- **Metrics:** Draft completion %; continuation activation %; answer-change %; accuracy vs tokens.
- **Success Criterion:** Provide a clear threshold regime (e.g., completion < 70%) where continuation becomes meaningfully active.
- **Estimated Cost/Time:** 2–4 days.
- **Expected Paper-Quality Gain:** High (turns diagnosis into validated boundary condition).

P1-E2 — Continuation causal effect decomposition

- **Target Claim:** "Continuation rarely changes answers in the current regime; overhead dominates."
- **Hypothesis:** Conditional on activation, answer-change is rare and/or not helpful.
- **Minimal Design:** For each problem, record whether continuation consumed tokens, whether final answer changed vs Stage-1 vote, and whether the change improved correctness.
- **Controls/Baselines:** A 'no-continue' ablation (stage-1 majority vote only) under same B, T_{draft}.
- **Metrics:** Activation %, flip %, flip-improves %, flip-hurts %.
- **Success Criterion:** Mechanistic explanation is numerically supported.
- **Estimated Cost/Time:** 1–2 days.
- **Expected Paper-Quality Gain:** Medium–High.

P2-E1 — One extra setting for generality (choose one)

- **Target Claim:** "The boundary condition generalizes beyond one model/dataset."
- **Hypothesis:** Draft completion depends on model + benchmark difficulty; DCS may activate more on longer-horizon settings.
- **Minimal Design:** Either (i) run on one harder/longer benchmark or length-stratified MATH subset, or (ii) run one additional math-reasoning model.
- **Controls/Baselines:** Same as main.
- **Metrics:** Same + completion/activation.
- **Success Criterion:** Show whether the negative result is regime-specific.
- **Estimated Cost/Time:** 3–7 days.
- **Expected Paper-Quality Gain:** Medium.

ASCII Diagram — Experiment Upgrade Plan

```
P0: Fairness
  Matched calls/tokens -> frontier plot

P1: Mechanism boundary
  T_draft sweep -> completion/activation
  activation -> answer-change -> correctness effect

P2: Generality
  1 extra benchmark/model -> verify boundary transfer
```

Novelty Verification & Related-Work Matrix

(9A) Contribution Novelty Verdict Board

Claim ID	Author Contribution Claim	Key Evidence Papers [n]	Novelty Verdict Tag	Why	Confidence	Required Repositioning
C1	**DCS method:** two-stage self-consistency that samples drafts, votes on interim answers, then continues high-vote branches with a hedge allocation (Page 2 - Method)	[5][6][9][4]	`partially_overlapping`	Prior work already uses *partial-trace signals* to guide/weight/prune self-consistency (prefix-guided sampling, hypothesis pruning, regeneration-based reliability, though pruning). DCS's specific novelty is the *interim-answer histogram + continuation-of-existing-drafts + hedge rule*, but the overall "use early signals to allocate later compute" axis is well populated.	Medium	Frame DCS as a **minimal continuation-based baseline** and focus the contribution on (i) the explicit continuation formulation (vs resampling/pruning) and (ii) the negative-result boundary condition rather than claiming a new paradigm.

Claim ID	Author Contribution Claim	Key Evidence Papers [n]	Novelty Verdict Tag	Why	Confidence	Required Repositioning
C2	**Empirical study / negative result:** on MATH-500 + Qwen2.5-Math-7B-Instruct, DCS matches accuracy but increases tokens; CGES-LNS Pareto-dominates (Page 4 - Main Results)	[2][1][8]	supported	The specific measurement that this continuation-based budgeting instantiation is not competitive under the evaluated regime is new (though it should be strengthened by matched-budget settings). Prior work provides baselines and evaluation perspective, but not this DCS case study.	Medium	Tighten to "in the evaluated setting" and add matched-budget curves so the negative result becomes a robust, reusable reference point.
C3	**Mechanistic diagnosis:** high draft completion (~96% at $T_{draft}=1024$) causes continuation to rarely activate, so DCS collapses to SC+overhead (Page 4 - Analysis; Page 5 - Discussion)	[3][10][8]	`partially_overlapping`	Related work already documents redundancy/answer convergence/overthinking under larger compute budgets and motivates early exit. Your diagnosis is a concrete instantiation for DCS, but the general phenomenon is not unique.	Medium	Position as: "**DCS is an empirical probe** of when continuation helps; we quantify a regime where it cannot help because drafts already terminate," and support it with activation/answer-change analyses and a T_{draft} sweep.

(9B) Related-Work Taxonomy Matrix

Taxonomy Layer	Branch/Leaf	Representative Papers [n]	Common Assumptions	Difference vs This Paper	Novelty Risk Signal
Root	Test-time compute for SC-style LLM reasoning	[8][7]	Compare methods under compute budgets (tokens/calls/latency proxies)	This paper is a single-method case study; needs matched-budget evaluation	Risk: "dominance" claims are fragile without budget-aware curves
Branch 1	Early stopping / adaptive halting		Answer-level signals can indicate "enough samples/steps"	DCS continues instead of halting	Medium overlap in "use early signals" axis
Leaf 1.1	Vote-entropy / window stopping	[1]	Vote distribution stabilizes early	DCS reallocates compute rather than stopping	Low–Medium
Leaf 1.2	Bayesian confidence posterior stopping	[2]	Confidence can be treated as evidence for answers	DCS uses interim answers (not token-prob confidence posteriors)	Medium
Leaf 1.3	Answer-convergence stopping	[3]	Answers converge before full reasoning ends	DCS attempts to exploit interim answers by continuing branches	Medium
Branch 2	Pruning / selection among hypotheses during SC		Intermediate structure reveals redundancy	DCS selects a few drafts to continue; pruning methods remove redundant ones	Medium
Leaf 2.1	Thought-similarity pruning	[4]	Similar "thoughts" imply redundant computation	DCS does not compute similarity; uses interim-answer votes	Medium
Leaf 2.2	Confidence+coverage pruning	[6]	A subset of hypotheses covers diversity while preserving confidence	DCS keeps fixed B drafts then continues k	Medium
Branch 3	Prefix / partial-trace reuse		Partial traces contain reusable information	DCS is closest here	High

Taxonomy Layer	Branch/Leaf	Representative Papers [n]	Common Assumptions	Difference vs This Paper	Novelty Risk Signal
Leaf 3.1	Prefix-guided resampling (prefix enhancement)	[5]	A good prefix can guide later generation	DCS continues *existing drafts* rather than resampling from a selected prefix	High overlap risk (closest neighbor)
Leaf 3.2	Regeneration-based prefix reliability weighting	[9]	Correct traces reproduce answers under regeneration more often	DCS uses interim-answer vote + continuation budget rather than regeneration frequency weights	High overlap risk
Branch 4	Compute-scaling / redundancy analyses		More compute can have diminishing/negative returns	This paper’s diagnosis should connect to these with concrete activation stats	Medium
Leaf 4.1	Budget-aware evaluation frameworks	[8]	Fairness requires compute normalization	Current paper still lacks matched budgets	High risk to claims until fixed
Leaf 4.2	Fair TTC evaluation protocol	[7]	Standardized prompting/extraction improves comparability	Your parser definition is currently missing	Medium
Leaf 4.3	Overthinking / diminishing returns	[10]	Extra tokens can reduce marginal utility	Supports your “drafts too long” narrative	Low–Medium

(9C) Head-to-Head Comparison Matrix

Ref [n]	Problem/Setting	Method Core	Strongest Overlap Point	Clear Difference	Impact on Final Judgment
[5]	Efficient SC-style inference	Use early confidence/answers to pick a promising **prefix** and guide later branches	Early-stage signal steers later compute	Prefix-guided **resampling** vs DCS **continuation of existing drafts**	Raises overlap risk for C1; related-work positioning must be explicit

Ref [n]	Problem/Setting	Method Core	Strongest Overlap Point	Clear Difference	Impact on Final Judgment
[9]	Reliability/efficiency for SC-style CoT	**Truncate + regenerate** to measure prefix-consistency reliability; reweight votes	Uses partial traces + continuation/regeneration	DCS allocates tokens via interim-answer vote; not regeneration-frequency weighting	Another close neighbor; pushes C1 toward `partially_overlapping`
[6]	Token-efficient SC	Periodically prune hypotheses using confidence + coverage (set cover)	Uses intermediate signals to allocate compute	Pruning reduces hypothesis count; DCS keeps drafts then continues k	Overlap in goal; DCS needs clearer differentiation
[4]	Efficient SC	Thought-level similarity pruning of redundant chains	Uses intermediate structure to reduce wasted compute	Similarity-based pruning vs vote-based continuation	Overlap in "reuse intermediate info" axis
[2]	Efficient SC	Bayesian posterior over answers; stop when posterior mass high	Budget-aware decision to stop sampling	DCS continues instead of halting; also uses different signals	Strong baseline that motivates matched-budget evaluation
[1]	Efficient SC	Window/vote-entropy early stopping	Stops when votes converge	DCS reallocates tokens to continuations	Supports baseline selection; not a novelty threat
[3]	Efficient CoT	Stop when answer converges; learn-to-stop variants	Recognizes redundancy in long CoT	DCS's diagnosis echoes redundancy findings	Makes C3 `partially_overlapping`
[8]	Budget-aware evaluation	Shows complex strategies often look good only because they spend more compute	Budget normalization emphasis	DCS should be evaluated as an allocation policy under fixed budgets	Supports revisions; increases scrutiny of current "Pareto" wording
[10]	Compute scaling analysis	Overthinking at high budgets; diminishing returns	"More tokens can hurt / waste"	DCS is a concrete case where extra stage adds overhead	Supports discussion but does not validate your mechanism without activation stats

Contribution-level Novelty Conclusion

Overall, **C1 is partially overlapping** with existing partial-trace reuse / pruning / prefix-guided self-consistency methods, so the paper should avoid broad novelty claims

and instead emphasize the specific continuation-based instantiation and hedged allocation rule. **C2 is supported** as a new empirical case study/negative result on DCS (but must be strengthened by matched-budget curves). **C3 is partially overlapping** with prior redundancy/early-exit insights; its value is strongest when presented as a *quantified boundary condition* for continuation-based budgeting, backed by activation and answer-change analyses.

ASCII Diagram — Related-Work Taxonomy Tree (Layered)

```
Related Work Taxonomy (Root: Test-time compute for SC-style reasoning)
├── Branch 1: Early stopping / adaptive halting
│   ├── Leaf 1.1: Vote-entropy / window stopping [1]
│   ├── Leaf 1.2: Bayesian confidence posterior stopping [2]
│   └── Leaf 1.3: Answer-convergence stopping [3]
├── Branch 2: Hypothesis pruning within SC
│   ├── Leaf 2.1: Thought-similarity pruning [4]
│   └── Leaf 2.2: Confidence+coverage pruning [6]
├── Branch 3: Prefix / partial-trace reuse
│   ├── Leaf 3.1: Prefix-guided resampling (prefix enhancement) [5]
│   └── Leaf 3.2: Regenerate-from-prefix reliability weighting [9]
└── Branch 4: Budget-aware evaluation & compute-scaling analyses
    ├── Leaf 4.1: Budget-aware evaluation frameworks [8]
    ├── Leaf 4.2: Fair TTC evaluation protocol [7]
    └── Leaf 4.3: Overthinking / diminishing returns analyses [10]

Position of this paper (DCS):
- Closest to Branch 3 (partial-trace reuse), but allocates compute by continuing a
subset of *existing
drafts*
  selected via interim-answer voting (not by resampling from a shared prefix or by
pruning).
- Primary value (as written) is a boundary-condition negative result: continuation
adds overhead when
drafts already terminate.
```

References

[1] Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning 2401.10480

[2] CGES: Confidence-Guided Early Stopping for Efficient and Accurate Self-Consistency 2511.02603

[3] Answer Convergence as a Signal for Early Stopping in Reasoning 2506.02536

[4] Slim-SC: Thought Pruning for Efficient Scaling with Self-Consistency 2509.13990

[5] Path-Consistency with Prefix Enhancement for Efficient Inference in LLMs 2409.01281

[6] Confidence-Weighted Token Set Cover for Early Hypothesis Pruning in Self-Consistency 2508.03979

[7] FEval-TTC: Fair Evaluation Protocol for Test-Time Compute 2511.01203

[8] Reasoning in Token Economies: Budget-Aware Evaluation of LLM Reasoning Strategies 2406.06461

[9] Reliable Chain-of-Thought via Prefix Consistency 2605.07654

[10] When More Thinking Hurts: Overthinking in LLM Test-Time Compute Scaling 2604.10739

Scores

Final Score: **4.5/10**.

Post-Revision Target: **[6.5, 7.5]/10**.

Rationale (score-driving factors):

- **Research value / insight:** The negative-result direction is useful, but currently under-validated because the key boundary condition (draft length vs solution length) is not experimentally swept.
- **Novelty:** The continuation-based instantiation has partial overlap with prefix/partial-trace reuse and pruning families; novelty is acceptable if repositioned as a minimal baseline + boundary-condition study.
- **Soundness / reproducibility:** Missing interim-answer parser and budget-matched comparisons are major issues for a compute-efficiency paper.
- **Fixability:** High—most deficits can be addressed with clear definitions plus a small set of targeted experiments and plots.

Appendix · Original Paper PDF

The following pages preserve source layout and include review annotations when available.

The appendix below attaches source paper pages and overlays review highlights for ISSUE / SUGGESTION / EVIDENCE objects.

Callout boxes are placed near page margins to avoid covering core content and remain within page boundaries.

End of report · DeepScientist Review Export

DRAFT-AND-CONTINUE SELF-CONSISTENCY: AN EMPIRICAL STUDY OF TWO-STAGE BRANCH BUDGETING FOR LLM REASONING

FARS
Analemma
fars@analemma.ai

ABSTRACT

Self-consistency improves large language model reasoning by sampling multiple chain-of-thought paths and aggregating via majority vote, but incurs substantial token costs from generating complete solutions. We investigate whether adaptive compute allocation can improve efficiency by identifying promising reasoning paths early and focusing continuation budget on them. We propose Draft-and-Continue Self-Consistency (DCS), a two-stage approach that samples short drafts, computes vote histograms over interim answers, and continues high-vote branches with additional tokens. Experiments on MATH-500 with Qwen2.5-Math-7B-Instruct reveal a negative result: DCS achieves 76.8% accuracy, matching baselines, but uses 19.6% more tokens than standard self-consistency. Simpler than confidence-guided early stopping (CGES-LNS), which coordinates DCS. Analysis shows that approximately 96% of drafts converge to the majority answer before the draft token limit, causing the continuation mechanism to rarely activate. Our findings demonstrate that simpler early-stopping methods outperform two-stage branch budgeting when draft lengths are sufficient for most problems.

WARNING: This paper was generated by an automated research system. The code is publicly available.[1]

1 INTRODUCTION

Scaling test-time compute has emerged as a powerful paradigm for improving large language model reasoning (Zhu et al., 2025). Self-consistency (Wang et al., 2022) exemplifies this approach by sampling multiple chain-of-thought reasoning paths (Wei et al., 2022) and aggregating answers via majority vote, achieving substantial accuracy gains over single-sample inference. However, this effectiveness comes at significant computational cost: generating multiple complete reasoning chains requires proportionally more tokens, making self-consistency expensive.

Recent work has sought to reduce this cost through early stopping strategies. Confidence-guided early stopping (Aghazadeh et al., 2025) terminates sampling when the probability of the majority answer exceeds a threshold. Slim-SC (Hong et al., 2025) prunes redundant reasoning chains online using thought embedding similarity. ESC (Li et al., 2024) applies early stopping based on window vote entropy. While effective, these approaches fundamentally discard partial reasoning—they stop sampling entirely rather than leveraging the information contained in incomplete chains. This raises a natural question: can we identify promising reasoning paths early and focus continuation budget on them?

We investigate this question through Draft-and-Continue Self-Consistency (DCS), a two-stage approach that attempts to improve token efficiency by allocating continuation budget to promising branches identified through early voting signals. In Stage 1, DCS samples $B = 7$ draft solutions with a token limit $T_{\text{draft}} = 1024$, extracts interim answers, and computes a vote histogram. In Stage 2, it selects $k = 3$ branches from high-vote answers and continues them for an additional $T_{\text{cont}} = 1024$ tokens, using a hedge allocation strategy that reserves one slot for the second-place answer to guard against early consensus errors.

[1] https://gitlab.com/fars-a/aletheia-adaptive-branch-allocation

Index ID: #P01-A01
Full text moved to continuation pages.

Index ID: #P01-A02
Full text moved to continuation pages.

Index ID: #P01-A03
Full text moved to continuation pages.

Index ID: #P01-A04
Full text moved to continuation pages.

Annotation Continuation · Source Page 1

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P01-A01 · SUGGESTION · MAJOR · #R001 · Part 1

The abstract already communicates the "negative result" clearly, but it currently over-generalizes the takeaway beyond what is directly tested (single model + single benchmark + one draft budget). Phrases like "Our findings demonstrate that simpler early-stopping methods outperform two-stage branch budgeting when draft lengths are sufficient for most problems" read like a general theorem, yet the paper later acknowledges limited scope. This mismatch can trigger reviewer pushback even if the empirical finding is valid.

Two clarifications would materially improve defensibility: (i) define what "token cost" means (generated output tokens only? includes prompt? includes both draft+continuation?) and (ii) make the *activation condition* explicit in the abstract (the continuation stage rarely activates because drafts already finish under $T_{draft}=1024$). This turns the narrative from "DCS is worse" into "DCS collapses to SC+overhead under this regime," which is a more scientific negative result.

Actionable rewrite: tighten to a 4–5 sentence abstract with explicit scope qualifiers and one sentence stating the "when does continuation help?" implication. Also avoid calling $T_{draft}=1024$ "short" unless you justify why it is short relative to typical MATH-500 solution length.

Mentor Revised Version:
Self-consistency improves LLM reasoning by sampling multiple chain-of-thought solutions and majority voting, but it increases inference cost due to repeated full-length generations. We study whether test-time compute can be allocated more efficiently by using early signals from partial solutions to decide which reasoning branches to extend. We evaluate a two-stage method, Draft-and-Continue Self-Consistency (DCS), which samples B draft solutions up to T_{draft} tokens, aggregates interim answers, and continues k selected drafts for an additional T_{cont} tokens. On MATH-500 with Qwen2.5-Math-7B-Instruct at $T_{draft}=1024$ and $T_{cont}=1024$, DCS matches the accuracy of uniform self-consistency (76.8%) but uses more generated tokens than both uniform self-consistency (+19.6%) and confidence-guided early stopping (+109%). We find that ~96% of drafts already complete within T_{draft}, so continuation rarely changes outcomes and mainly adds overhead; this suggests that continuation-based branch budgeting is only promising when the draft budget is small relative to typical solution length.

References:
[1] Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning 2401.10480

[2] CGES: Confidence-Guided Early Stopping for Efficient and Accurate Self-Consistency 2511.02603

#P01-A02 · SUGGESTION · MAJOR · #R002 · Part 1

This opening paragraph frames "test-time compute scaling" broadly, but the rest of the paper evaluates a very specific sub-case: self-consistency-style *multi-sample* CoT with majority voting, measured by generated tokens / API calls on MATH-500. Right now, readers must infer that scope and metric, which weakens the motivation-to-method alignment.

Two concrete fixes:
1) Define the resource axis *up front*: are you optimizing (a) generated output tokens, (b) number of model calls, (c) wall-clock latency, or (d) dollar cost? The experiments later report tokens and calls; the intro should commit to these as the primary efficiency metrics.
2) Narrow the claim "Scaling test-time compute … paradigm" to the sampling/aggregation family you actually study, so the reader does not expect coverage of search-based methods (ToT/MCTS) or verification-heavy methods.

Mentor Revised Version:
Test-time compute scaling improves LLM reasoning by spending additional inference resources—typically extra model calls and generated tokens—to explore multiple candidate solutions and aggregate them. In this paper, we focus on self-consistency-style sampling with majority voting, where the dominant cost comes from generating multiple full chain-of-thought traces. This motivates a budget-allocation question: under a fixed per-problem inference budget (measured in generated tokens and model calls), can we redistribute compute across sampled reasoning paths to reduce cost without sacrificing accuracy?

#P01-A03 · SUGGESTION · MAJOR · #R003 · Part 1 · CONT.

This paragraph's key move is the gap claim: early-stopping/pruning methods "fundamentally discard partial reasoning" and thus do not "leverage the information contained in incomplete chains." This is rhetorically strong, but it is likely overstated: several efficiency methods *do* reuse intermediate signals (e.g., prefix-guided generation / thought-level pruning) rather than purely stopping.

Because the paper's core contribution is a *continuation* mechanism, the introduction needs a more careful gap statement: (i) which signals prior methods use (confidence, entropy, similarity), (ii) what they do with partial reasoning (stop vs prune vs guide next samples), and (iii) what is genuinely missing (e.g., allocating *additional tokens* to specific branches conditional on interim answers).

Annotation Continuation · Source Page 1 (cont. 2)

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P01-A03 · SUGGESTION · MAJOR · #R003 · Part 2

Concrete repair: replace the “fundamentally discard” sentence with a narrower statement, and add one short sentence acknowledging prefix-guided or pruning-based SC variants. This makes DCS’s novelty axis clearer and avoids a reviewer flagging the gap as incorrect.

Mentor Revised Version:
Prior work reduces the cost of self-consistency by adaptively deciding *how many* samples to draw (early stopping) or by pruning redundant samples online using lightweight signals such as answer confidence, vote entropy, or inter-chain similarity. These methods primarily save compute by stopping or pruning, rather than by reallocating additional generation budget to specific branches conditioned on their partial outputs. This raises a complementary question studied here: can interim answers from partial solutions be used to select a small subset of branches to extend with additional tokens, improving the accuracy–cost trade-off under a fixed token budget?

References:
[1] Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning 2401.10480

[2] Slim-SC: Thought Pruning for Efficient Scaling with Self-Consistency 2509.13990

[3] Path-Consistency with Prefix Enhancement for Efficient Inference in LLMs 2409.01281

#P01-A04 · SUGGESTION · MAJOR · #R004 · Part 1

This paragraph introduces DCS with concrete hyperparameters, but several implementation-critical details are still implicit:

1) Interim answer extraction: For MATH-500, answers can appear in different formats (boxed, final line, multiple candidates). If interim answers are extracted from partial drafts, the extraction rule matters a lot (and affects the NULL bucket rate). A reader cannot reproduce DCS without a precise extractor definition and examples.

2) “Drafts already finished” edge case: Your own results later show ~96% drafts complete within T_{draft}. In that regime, what does “continue” mean operationally? Do you (a) force the model to continue generating beyond a finished answer, (b) re-prompt to ‘continue reasoning’, or (c) do nothing and just count it as completed? Each choice changes both token accounting and the claimed mechanism.

3) Cost definition: you mention token limits, but not whether tokens include the prompt, system message, and tool overhead (if any). Since the paper’s main claim is efficiency, cost accounting must be explicit.

Actionable fix: add a short “Algorithm 1: DCS” box right after this paragraph that specifies: sampling prompts, stopping conditions, answer extractor, selection rule, continuation prompting, and token/call accounting. Also include an explicit ‘draft-completed’ branch (which seems to dominate in your setting).

Mentor Revised Version:
We investigate Draft-and-Continue Self-Consistency (DCS), a two-stage inference procedure that reallocates a fixed sampling budget across reasoning branches based on interim answers. In Stage 1, we sample B draft responses up to T_{draft} generated tokens and extract an interim answer from each partial trace using a deterministic parser (Appendix X), assigning unparseable drafts to a NULL bucket. We then form a vote histogram over interim answers and select k drafts to extend, using a hedge rule that allocates $k{-}1$ continuations to the top-voted answer and one continuation to the second-voted answer when available. In Stage 2, each selected draft is continued for up to T_{cont} additional generated tokens (unless it already terminated before T_{draft}, in which case no continuation tokens are generated). The final prediction is the majority vote over the completed outputs of the selected continuations, and we report efficiency in terms of total generated tokens and total model calls per problem.

Our experiments on MATH-500 with Qwen2.5-Math-7B-Instruct reveal a negative result: DCS matches baseline accuracy (76.8%) but uses 19.6% more tokens than standard self-consistency and 109% more than CGES-LNS, which Pareto-dominates DCS on the accuracy-efficiency frontier. Analysis reveals that approximately 96% of drafts complete [illegible] causing the continuation mechanism to rarely activate meaningfully. Our contributions are:

- We introduce Draft-and-Continue Self-Consistency (DC[illegible] tempts to improve efficiency by continuing promising branches rather than stopping early.
- We conduct rigorous experiments on MATH-500 comparing DCS against greedy decoding, uniform self-consistency, and confidence-guided early stopping.
- We report a negative result with detailed analysis: DCS fails to improve efficiency because high draft completion rates (∼96%) undermine the continuation mechanism, demonstrating that simpler early-stopping methods dominate two-stage approaches when draft budgets are sufficient.

Index ID: #P02-A01
Full text moved to continuation pages.

2 METHOD

2.1 PROBLEM SETUP

Self-consistency (Wang et al., 2022) improves chain-of-thought reasoning (Wei et al., 2022) by sampling multiple reasoning paths and aggregating their answers via majority vote. Given an input problem x, the method samples n independent reasoning chains $\{(r_i, a_i)\}_{i=1}^{n}$, where r_i denotes the reasoning path and a_i the extracted answer. The final prediction is determined by majority vote:

$$\hat{a} = \arg\max_{a} \sum_{i=1}^{n} \mathbf{1}(a_i = a) \tag{1}$$

While effective, this approach requires generating n complete sol[illegible] when reasoning chains are long. The key question we investigate is whether compute can be allocated more efficiently by identifying promising reasoning paths ea[illegible]

Index ID: #P02-A02
Full text moved to continuation pages.

2.2 DRAFT-AND-CONTINUE SELF-CONSISTENCY

We propose Draft-and-Continue Self-Consistency (DCS), a two-stage approach that attempts to improve token efficiency by allocating continuation budget to promising branches identified through early voting signals. Figure 1 illustrates the overall architecture.

Stage 1: Draft Sampling. Given an input problem x, we sample B independent draft solutions, each with a token limit T_{draft}. For each draft i, we extract an interim answer a_i^{draft} from the partial reasoning. Drafts that do not produce a parseable interim answer are assigned to a special `NULL` bucket and excluded from continuation selection.

Vote Histogram. We compute vote counts over the interim answ[illegible]

$$c(a) = \sum_{i=1}^{B} \mathbf{1}(a_i^{\text{draft}} = a) \tag{2}$$

Let $a^{(1)}$ and $a^{(2)}$ denote the top-1 and top-2 answers by vote coun[illegible], respectively.

#P02-A03 · ISSUE · MAJOR · #R007 · INDEX · CONT.
Index ID: #P02-A03
Full text moved to continuation pages.

Stage 2: Continuation Allocation. We select k branches to continue for an additional T_{cont} tokens. To hedge against the "majority-wrong" failure mode where the early consensus is incorrect, we employ a hedge allocation strategy: if a second-place answer $a^{(2)}$ exists, we allocate $k-1$ continuation slots to branches from $a^{(1)}$ and 1 slot to a branch from $a^{(2)}$. If all drafts agree on a single answer, all k slots go to branches from that answer. Branches within [illegible] arbitrarily.

Index ID: #P02-A04
Full text moved to continuation pages.

Annotation Continuation · Source Page 2

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P02-A01 · SUGGESTION · MAJOR · #R005 · Part 1

This paragraph ends with “Our contributions are:” but the PDF snippet that follows immediately jumps to “2 METHOD” (no bullet list). If the contributions list exists elsewhere, it is not present here; if it is missing, the paper loses a key signpost and the narrative feels like “negative result” is the only contribution.

Even for a negative-result paper, it helps to separate (i) the methodological proposal (DCS as a two-stage compute-allocation scheme), (ii) the empirical finding (it is Pareto-dominated under this regime), and (iii) the diagnostic explanation (draft completion rate collapses the mechanism) as three explicit bullets. This also prevents the method section from being forced to carry “contribution framing” burden.

Actionable fix: insert a compact 3-bullet contribution list right here before Section 2, then keep the Method section purely about the algorithm.

Mentor Revised Version:
Our experiments on MATH-500 with Qwen2.5-Math-7B-Instruct produce a negative result: under the default budget, DCS matches self-consistency accuracy but increases token cost, and confidence-guided early stopping Pareto-dominates DCS on the accuracy–cost frontier. We summarize three contributions:
(1) DCS: a two-stage self-consistency variant that uses interim-answer voting to select a small set of branches to continue under a fixed budget;
(2) Budget-aware evaluation against strong efficiency baselines (uniform self-consistency and confidence-guided early stopping) reporting accuracy, generated tokens, and calls;
(3) Diagnostic analysis showing that high draft completion rates (~96% at $T_{draft}=1024$) collapse the continuation mechanism, clarifying when continuation-based branch budgeting is unlikely to help.

#P02-A02 · SUGGESTION · MAJOR · #R006 · Part 1

Section 2.1 defines standard self-consistency and majority vote, but it does not formalize the optimization target of this paper. DCS is fundamentally a *budget allocation* policy, so you need at least one formal objective statement to avoid the method feeling heuristic.

Concrete suggestion: introduce a budget-aware objective such as “maximize accuracy subject to expected generated tokens $\leqslant$ B” or “minimize expected tokens subject to accuracy $\geqslant$ A,” *and clarify which one your experiments operationalize. Right now, you report both accuracy and average tokens, but there is no explicit objective tying them together.*

Also, since you compare to CGES (adaptive number of calls) and uniform SC (fixed n), the notion of “budget” should include both number of calls and per-call max tokens. A simple formalization can cover both.

Mentor Revised Version:
Given an input problem x, self-consistency samples n independent chains and predicts by majority vote (Eq. 1). In this work we study budget-aware variants of self-consistency. Let C denote inference cost measured by total generated tokens (and equivalently the number of model calls when per-call limits are fixed). Our goal is to design an inference policy pi that allocates computation across candidate reasoning paths to maximize expected accuracy under a fixed cost budget, i.e., $max_{pi};mathbb{E}[mathrm{Acc}(pi(x))]$ subject to $mathbb{E}[C(pi(x))] le mathcal{B}$. DCS (Section 2.2) instantiates pi by using interim-answer votes from partial drafts to decide which branches to continue, and our experiments report the resulting accuracy–cost trade-offs against standard self-consistency and adaptive early-stopping baselines.

#P02-A03 · ISSUE · MAJOR · #R007 · Part 1 · CONT.

This paragraph introduces two implementation-critical operations—(i) extracting an interim answer from a partial draft, and (ii) assigning non-parseable drafts to a NULL bucket—but the paper does not specify the exact extraction rule. On MATH-style problems, small changes in answer parsing (boxed vs last line, fractions vs decimals, multiple candidate answers, units, etc.) can materially change vote histograms, NULL rates, and continuation choices.

Why this is major: your main explanation for the negative result is that ~96% drafts “complete” within T_{draft}. But without defining what counts as a “parseable final answer” vs an “interim answer,” readers cannot validate whether this completion rate is a property of the model/problem or an artifact of the parser (e.g., extracting an answer too early).

Concrete fix (must): add a deterministic answer-extraction specification, ideally as a short subsection or appendix with:
- exact regex / parsing logic (and how you handle multiple matches),
- normalization (strip spaces, simplify fractions, handle pm, etc.),
- examples of partial traces where an interim answer exists vs not,
- definition of “draft complete” (did generation stop naturally, or did you detect an answer marker?),
- how NULL drafts affect $a^{(1)}, a^{(2)}$ selection.

Annotation Continuation · Source Page 2 (cont. 2)

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P02-A03 · ISSUE · MAJOR · #R007 · Part 2

Minimal acceptance criterion: given one raw model completion, an independent reader should be able to compute a_i^{draft}, decide NULL vs non-NULL, and replicate vote counts.

Mentor Revised Version:
Stage 1 samples B draft responses up to T_{draft} generated tokens. We extract an interim answer a_i^{draft} using a deterministic parser (Appendix X) that (i) prioritizes the last occurrence of a "boxed{cdot}" answer; (ii) otherwise falls back to the final line matching a numeric expression; and (iii) normalizes equivalent formats (fractions/decimals, sign, whitespace). Drafts with no parseable answer are assigned to a NULL bucket and are excluded from continuation selection. We define a draft as "complete" if the model emits an explicit end-of-answer marker (e.g., a boxed final answer) before hitting T_{draft}; otherwise it is "truncated" and eligible for continuation.

#P02-A04 · SUGGESTION · MAJOR · #R008 · Part 1

Stage 2's hedge allocation is a reasonable idea, but the statement "Branches within each answer cluster are selected arbitrarily" is problematic for two reasons:

1) Reproducibility/variance: arbitrary selection (especially if randomized) can add variance across seeds beyond what is inherent to sampling. You later report higher variance for DCS; uncontrolled intra-cluster selection can contribute directly to that.

2) Fairness of comparison: if baseline SC uses a fixed n and you use arbitrary selection among drafts, the resulting compute distribution across solution qualities can shift in a way that is not attributable to the proposed mechanism.

Concrete fixes:
· Replace "arbitrarily" with a deterministic selection rule (e.g., earliest-finished, highest mean token logprob, longest prefix without answer yet, or simply "first occurrence" in sampling order) and document it.
· Add an ablation: random selection vs deterministic selection, and (if available) a score-based selection, to isolate how much of the effect is from hedge allocation vs selection noise.

Mentor Revised Version:
Within each interim-answer cluster, we select continuation candidates using a deterministic rule (sampling order) to avoid additional randomness beyond the draft sampling itself. In an ablation, we also evaluate random intra-cluster selection to quantify how much variance it introduces relative to deterministic selection, keeping B,k,T_{draft},T_{cont} fixed.

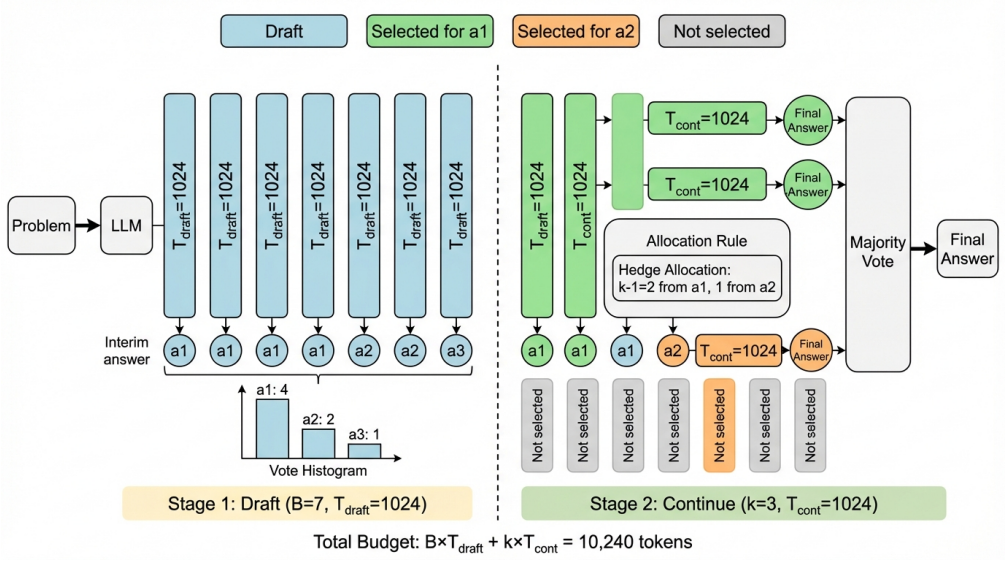


Figure 1: Overview of Draft-and-Continue Self-Consistency (DCS). Stage 1 samples $B = 7$ drafts with $T_{\text{draft}} = 1024$ tokens, extracts interim answers via vote histogram, and selects $k = 3$ branches. Stage 2 continues selected branches with $T_{\text{cont}} = 1024$ additional tokens using hedge allocation. Final answer is determined by majority vote across all completed responses.

Final Aggregation. After continuation, we extract final answers from all continued branches and apply majority vote (Equation 1) to determine the prediction. The total token budget is approximately $B \cdot T_{\text{draft}} + k \cdot T_{\text{cont}}$.

#P03-A01 · ISSUE · MAJOR · #R009 · INDEX · CONT.
Index ID: #P03-A01
Full text moved to continuation pages.

2.3 Hyperparameters

We use the following default hyperparameters: $B = 7$ drafts, $k = 3$ continuations, $T_{\text{draft}} = 1024$ tokens, and $T_{\text{cont}} = 1024$ tokens. This yields a total budget of approximately $7 \times 1024 + 3 \times 1024 = 10{,}240$ tokens per problem. We employ hedge allocation mode, which reserves one continuation slot for the second-place answer when available. All sampling uses temperature 0.8 and top-p 0.95 to encourage diversity in reasoning paths.

#P03-A02 · ISSUE · MAJOR · #R010 · INDEX · CONT.
Index ID: #P03-A02
Full text moved to continuation pages.

3 Experiments

3.1 Experimental Setup

Dataset. We evaluate on MATH-500 (Hendrycks et al., 2021), a subset of 500 competition-style mathematics problems requiring multi-step reasoning. Problems span diverse topics including algebra, geometry, number theory, and combinatorics, with exact-match evaluation.

Model. We use Qwen2.5-Math-7B-Instruct (Yang et al., 2024), a 7B parameter model fine-tuned for mathematical reasoning. This model achieves strong performance while avoiding ceiling effects that would obscure method differences.

Index ID: #P03-A03
Full text moved to continuation pages.

Baselines. We compare DCS against three baselines: (1) **Greedy CoT**: single-sample chain-of-thought with temperature 0 and max tokens 2048; (2) **Uniform SC**: standard self-consistency with $n = 5$ samples, temperature 0.8, and top-p 0.95; (3) **CGES-LNS** (Aghazadeh et al., 2025): confidence-guided early stopping with length-normalized scoring and confidence threshold $\gamma = 0.95$.

#P03-A04 · ISSUE · MAJOR · #R011 · INDEX · CONT.
Index ID: #P03-A04
Full text moved to continuation pages.

Annotation Continuation · Source Page 3

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P03-A01 · ISSUE · MAJOR · #R009 · Part 1

The text states: “The total token budget is approximately $Bcdot T_{draft} + kcdot T_{cont}$.” This is only an upper bound if every draft always runs to the token limit and every continuation always consumes T_{cont} tokens. But later you report that ~96% of drafts “complete naturally within the token limit,” implying many drafts stop early and (depending on your continuation policy) some continuations may be no-ops.

Why this matters: the paper’s central claim is efficiency. Using a worst-case budget expression without also reporting the *expected* realized token cost risks confusing readers and can create internal inconsistencies (e.g., “budget 10,240 tokens” vs Table 1’s ~4,114 avg tokens). This should be made explicit, not left to inference.

Concrete fix (must): replace the sentence with an expectation form and define the stopping variables, e.g.
$$mathbb{E}[C]=sum_{i=1}^{B}mathbb{E}[min(L_i, T_{draft})] + sum_{j=1}^{k}mathbb{E}[min(L'_j, T_{cont})],$$
where L_i is the natural length of draft i and L'_j is additional length after continuation prompting. Then explicitly connect this to the *measured* average tokens in Table 1.

Mentor Revised Version:
The worst-case token cap is $B,T_{draft}+k,T_{cont}$, but the realized cost is lower because drafts and continuations may terminate early. In our experiments we therefore report the average realized generated tokens per problem, and we interpret DCS as a budget-allocation policy whose effective behavior depends on the draft completion rate under T_{draft}.

#P03-A02 · ISSUE · MAJOR · #R010 · Part 1

This hyperparameter paragraph has two issues:

(1) Correctness/reproducibility: T_{cont} is rendered as $mathrm{i0 2 4}$ (likely a typo/extraction artifact for 1024). Any ambiguity in token limits undermines reproducibility.

(2) Evidence sufficiency for the main conclusion: the paper’s central explanation is that $T_{draft}=1024$ is large enough that drafts already finish (~96%), collapsing the continuation mechanism. If so, then the most direct supporting evidence is a controlled sweep over T_{draft} (e.g., 128/256/512/1024) holding total budget fixed, showing when continuation begins to matter and whether DCS ever becomes competitive.

Without at least a small sweep, the conclusion “two-stage branch budgeting is dominated when drafts are sufficient” remains plausible but under-validated: readers cannot tell if this is an artifact of one hyperparameter choice.

Concrete fix (must):
· Correct the T_{cont} symbol and ensure all hyperparameters are machine-readable.
· Add a T_{draft} sweep (and possibly B,k) with a fixed total token cap, reporting (i) draft completion rate, (ii) continuation activation rate, (iii) accuracy/tokens frontier.

Mentor Revised Version:
We use default hyperparameters $B{=}7$, $k{=}3$, $T_{draft}{=}1024$, and $T_{cont}{=}1024$ tokens. To test sensitivity to draft length, we additionally sweep $T_{draft}in{128,256,512,1024}$ while adjusting T_{cont} (or B,k) to keep the total token cap comparable, and we report how draft completion and continuation activation vary with T_{draft}.

#P03-A03 · SUGGESTION · MAJOR · #R012 · Part 1 · CONT.

The setup evaluates only MATH-500 with a single model. That is acceptable for a concise negative result, but it does not support broader statements about “two-stage branch budgeting” as a general approach. Your own discussion notes that harder benchmarks or shorter drafts may change conclusions; the empirical section should already include at least one such stress test.

Low-cost, high-yield additions:
· One “harder / longer-solution” setting: e.g., a subset of problems with longer gold solutions, or a harder split (if available), or a different benchmark where solution length often exceeds 1024.
· One “shorter draft” setting: sweep T_{draft} downward (128–512) and keep total budget fixed.
· One different model size/family (even one additional model) to show whether 96% completion is model-specific.

Mentor Revised Version:
We start with MATH-500 + Qwen2.5-Math-7B-Instruct to isolate the mechanism. To test whether the negative result is driven by draft length vs general properties of two-stage budgeting, we additionally evaluate (i) a shorter-draft regime ($T_{draft}le 512$) under matched total token caps,

Annotation Continuation · Source Page 3 (cont. 2)

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P03-A03 · SUGGESTION · MAJOR · #R012 · Part 2

and (ii) one harder benchmark or a length-stratified subset where solutions more often exceed the draft budget.

#P03-A04 · ISSUE · MAJOR · #R011 · Part 1

The baseline setup mixes methods with very different call budgets and token limits:

- Greedy: 1 call, max tokens 2048.
- Uniform SC: 5 calls, no per-call max stated here.
- CGES-LNS: up to 5 calls (adaptive).
- DCS: later reported as ~10 calls (7 drafts + 3 continuations).

As a result, comparisons of "Avg Tokens" and "Efficiency" can be confounded by budget mismatch rather than algorithmic differences. This is especially important because your paper is about *budget allocation*: fair evaluation typically requires either (a) fixed total token cap across methods, or (b) fixed max calls (or fixed dollar cost), then compare accuracy.

Concrete fix (must): add at least two matched-budget baselines:
1) Uniform SC with $n{=}10$ calls (to match DCS call budget), and
2) A DCS variant with $B{=}5$ (or $B{=}4,k{=}1$) so total calls match CGES/Uniform SC.
Additionally, report a fixed-token-cap evaluation where each method is constrained to the same expected generated tokens.

Also, CGES-LNS is mentioned but not defined in this paper; provide a one-sentence description of what is "length-normalized scoring" in your implementation and confirm whether you used the authors' official code.

Mentor Revised Version:
To ensure fair budget-aware comparison, we evaluate all methods under matched inference budgets. Specifically, we report (i) a matched-calls setting where Uniform SC uses $n{=}10$ samples and DCS uses $B{=}7,k{=}3$ (10 calls total), and (ii) a matched-token-cap setting where each method is run under the same expected generated-token budget. For CGES-LNS, we follow the reference implementation and specify the exact length-normalized confidence score used for posterior updates and stopping.

Table 1: Main results on MATH-500 with Qwen2.5-Math-7B-Instruct. DCS matches baseline accuracy but uses more tokens than both Uniform SC and CGES-LNS. Best in **bold**, second-best underlined. Δ_{Greedy} shows accuracy improvement over Greedy CoT. Δ_{Uniform} shows token change relative to Uniform SC.

Method	Accuracy (%)	Avg Tokens	Avg Calls	Δ_{Greedy}	Δ_{Uniform}	Efficiency
Greedy CoT	75.6 ± 0.28	**654.3**	**1.0**	—	-81.0%	**115.6**
Uniform SC	76.67 ± 0.41	3439.6	5.0	+1.1%	—	22.3
CGES-LNS	$\mathbf{76.8 \pm 0.30}$	1963.0	2.48	+1.2%	$\mathbf{-42.9\%}$	39.1
DCS (Ours)	76.8 ± 0.75	4114.5	$\sim$10	+1.2%	+19.6%	18.7

Table 2: DCS diagnostic metrics revealing why the method fails to improve efficiency. The high draft completion rate ($\sim$96.3%) indicates most drafts finish within $T_{\text{draft}} = 1024$ tokens, causing the continuation stage to rarely activate meaningfully.

Seed	Accuracy (%)	Avg Tokens	Fallback (%)	Draft Complete (%)
42	76.6	4091.4	4.0	96.0
123	76.0	4137.2	3.6	96.4
456	77.8	4114.9	3.6	96.4
Mean $\pm$ Std	$\mathbf{76.8 \pm 0.75}$	$\mathbf{4114.5 \pm 18.7}$	$\mathbf{3.73 \pm 0.19}$	$\mathbf{96.3 \pm 0.19}$

Evaluation Metrics. We report accuracy (exact match), average generated tokens per problem, and average API calls per problem. All experiments use three random seeds (42, 123, 456) and report mean $\pm$ standard deviation.

3.2 Main Results

Table 1 presents the main experimental results comparing all four methods on MATH-500.

DCS achieves 76.8% accuracy on MATH-500, matching both Uniform SC (76.67%) and CGES-LNS (76.8%) within statistical variance. This demonstrates that the two-stage draft-and-continue mechanism is functionally correct and produces competitive accuracy. However, DCS fails to deliver the hypothesized efficiency gains. DCS uses 4114.5 tokens per problem on average, which is 19.6% more than Uniform SC (3439.6 tokens) and 109% more than CGES-LNS (1963.0 tokens). Critically, CGES-LNS Pareto-dominates DCS on the accuracy-efficiency frontier: it achieves identical accuracy while using 52% fewer tokens and requiring only 2.48 API calls for DCS. Furthermore, DCS exhibits higher variance (0.75%) compared to all baselines, suggesting that the two-stage process introduces additional randomness rather than improving consistency.

3.3 Analysis: Why DCS Fails

To understand why DCS fails to improve efficiency, we examine diagnostic metrics in Table 2.

The key finding is that approximately 96.3% of drafts complete naturally within the $T_{\text{draft}} = 1024$ token limit, producing a parseable final answer without requiring continuation. This means the continuation stage—the core mechanism intended to improve efficiency by focusing compute on promising branches—rarely activates in a meaningful way. When drafts already contain complete solutions, continuing them provides no additional benefit while still incurring the overhead of the two-stage process. The fallback fraction of only 3.73% indicates that DCS effectively degenerates into a variant of standard self-consistency with extra overhead from the draft selection and continuation allocation steps. The higher variance (0.75% vs. 0.30–0.41%) further suggests that the two-stage process introduces instability rather than improving consistency.

3.4 Discussion

The failure of DCS reveals an important insight about two-stage branch budgeting approaches: their effectiveness depends critically on the relationship between draft length and problem complexity.

Index ID: #P04-A01
Full text moved to continuation pages.

Index ID: #P04-A02
Full text moved to continuation pages.

Index ID: #P04-A03
Full text moved to continuation pages.

Annotation Continuation · Source Page 4

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P04-A01 · SUGGESTION · MAJOR · #R013 · Part 1

This evaluation-metrics paragraph is a solid start (multi-seed mean±std, tokens, calls). Two upgrades would make the empirical claims more reviewer-proof:

1) Statistical testing: Accuracy differences are small (fractions of a percent). Mean±std over 3 seeds is helpful, but a paired test at the problem level (e.g., McNemar’s test for exact-match differences between methods) would better support “matches baseline accuracy” claims.

2) Cost model clarity: “Avg tokens” is reported, but readers often want wall-clock latency or dollar cost. Even if you cannot measure latency robustly, you can provide a simple cost proxy: total generated tokens + number of calls (which affects overhead/parallelism). At minimum, state explicitly that you measure *generated output tokens excluding prompts* (or including—just be explicit).

Mentor Revised Version:
We evaluate accuracy (exact match), average generated output tokens per problem, and average model calls per problem. Results are reported as mean±std over three random seeds. To quantify whether small accuracy deltas are meaningful, we additionally report a paired significance test (McNemar) on per-problem correctness differences between methods. For efficiency, we treat generated tokens as the primary cost proxy and separately report call count as a practical deployment constraint.

#P04-A02 · SUGGESTION · MAJOR · #R014 · Part 1

This main-results paragraph makes strong frontier language (“Pareto-dominates”) based on a single operating point. The arithmetic checks out given the reported token means, but the argument would be much more compelling if you showed the *curve* rather than one point.

Why this matters: For budget-allocation methods, a single configuration can be unrepresentative. Reviewers may reasonably ask whether DCS becomes competitive under different budgets (e.g., smaller T_{draft}) even if it loses at $T_{draft}=1024$. Presenting an accuracy–cost frontier (varying T_{draft} or total budget) is a standard way to support Pareto claims.

Concrete fixes:
· Add an accuracy vs expected-token plot where each method traces a curve over budgets (e.g., for Uniform SC vary n, for CGES vary $gamma$, for DCS vary T_{draft} and/or k under a fixed cap).
· Add matched-budget comparisons (equal calls or equal token caps) so the reader can separate “algorithmic” from “budget” effects.
· Define the “Efficiency” column in Table 1 explicitly (it appears to be accuracy per 1k tokens) and warn that ratio metrics can be misleading; keep it, but complement with frontier plots.

Mentor Revised Version:
Across MATH-500, DCS matches baseline accuracy but increases expected token cost under the default budget, and CGES-LNS achieves the same accuracy with substantially fewer tokens. To support this frontier claim beyond a single point, we additionally plot accuracy versus expected generated tokens across a range of budgets for each method (varying n for Uniform SC, $gamma$ for CGES, and T_{draft}/k for DCS under a fixed total cap). We also include matched-budget baselines (equal calls and equal expected-token caps) and define the ‘Efficiency’ metric as accuracy per 1k generated tokens.

#P04-A03 · SUGGESTION · MAJOR · #R018 · Part 1 · CONT.

This analysis paragraph provides a clear hypothesis for failure (high draft completion rate). To make it scientifically airtight, you need to show that completion is not only frequent but also *mechanistically* explains why continuation cannot help.

Concrete analyses to add (low cost):
· Continuation activation rate: fraction of problems where at least one selected branch was actually truncated at T_{draft} and thus received non-zero continuation tokens.
· Answer-change rate: among problems with non-zero continuation, how often does continuation change the final majority answer vs leaving it unchanged?
· Completion vs correctness correlation: are truncated drafts more likely to be wrong? If truncated drafts are rare but disproportionately wrong, continuation might still matter.
· Counterfactual: simulate a shorter draft budget (or artificially truncate) and re-run continuation to see if accuracy changes.

These analyses would directly connect “96% complete” to “continuation provides no value,” rather than relying on an intuitive explanation.

Mentor Revised Version:
We attribute DCS’s inefficiency to the high draft completion rate at $T_{draft}{=}1024$. To validate this mechanism, we will report (i) the

Annotation Continuation · Source Page 4 (cont. 2)

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P04-A03 · SUGGESTION · MAJOR · #R018 · Part 2

continuation activation rate (percentage of problems where selected branches actually consume continuation tokens), (ii) the answer-change rate conditional on activation, and (iii) the relationship between draft truncation and correctness. If continuation rarely activates and rarely changes answers when it does, this supports the conclusion that DCS collapses to self-consistency with added overhead in this regime.

With $T_{\text{draft}} = 1024$ tokens, Qwen2.5-Math-7B-Instruct can solve most MATH-500 problems completely, leaving little room for the continuation mechanism to provide value. This suggests that two-stage approaches may require either shorter draft limits or harder benchmarks where solutions genuinely require more tokens than the draft budget allows.

Our study has several limitations. We evaluate on a single dataset (MATH-500) with a single model (Qwen2.5-Math-7B-Instruct), and results may differ on other benchmarks or with other models. The hyperparameter choices ($B = 7$, $k = 3$, $T_{\text{draft}} = 1024$) were no[illegible] configurations might yield different results. Nevertheless, the core finding—that high draft completion rates undermine the continuation mechanism—is likely to ge[illegible]ize to similar settings where the draft budget is sufficient for most problems.

Index ID: #P05-A01
Full text moved to continuation pages.

4 Related Work

Self-Consistency and Variants. Self-consistency (Wang et al., 2022) improves chain-of-thought reasoning by sampling multiple reasoning paths and aggregating via majority vote. Adaptive-Consistency (Aggarwal et al., 2023) reduces cost by stopping sampling once the vote distribution stabilizes. Path-Consistency (Zhu et al., 2024) reuses high-confidence prefixes to guide subsequent sampling. Reasoning-Aware Self-Consistency (Wan et al., 2024) uses lightweight features to score reasoning paths and enable earlier stopping.

Efficient Self-Consistency. Several methods aim to reduce the computational cost of self-consistency. CGES (Aghazadeh et al., 2025) uses confidence-guided early stopping with Bayesian aggregation to terminate sampling when the posterior probability of the majority answer exceeds a threshold. Slim-SC (Hong et al., 2025) prunes redundant reasoning chains online using thought embedding similarity. ESC (Li et al., 2024) applies early stopping based on window vote entropy for multi-step reasoning. Optimal Self-Consistency (Feng et al., 2025) provides theoretically grounded adaptive sample allocation.

Tree-Based Reasoning and Test-Time Scaling. Tree of Thoughts (Yao et al., 2023) explores reasoning branches with evaluation-guided expansion. Graph of Thoughts (Besta et al., 2023) generalizes this to graph-based reasoning state transitions. LATS (Zhou et al., 2023) applies MCTS-style search with self-evaluation signals. Recent work on scaling test-time compute (Zhu et al., 2025) and adaptive inference (Manvi et al., 2024) explores dynamic resource allocation for LLM reasoning.

Positioning DCS. Our Draft-and-Continue Self-Consistency differs from early-stopping methods by attempting to continue promising branches rather than simply terminating sampling. While early-stopping approaches reduce cost by stopping when confidence is high, DCS hypothesizes that selectively extending high-vote branches could improve accuracy-efficiency trade-offs. Our negative result demonstrates that this continuation-based approach does not yield efficiency gains over simpler early-stopping methods, providing empirical evidence that branch continuation may be unnecessary when draft lengths are sufficient for most problems.

Index ID: #P05-A02
Full text moved to continuation pages.

5 Conclusion

We investigated Draft-and-Continue Self-Consistency (DCS), a two-stage approach that attempts to improve token efficiency by continuing promising reasoning branches identified through early voting signals. Our experiments on MATH-500 reveal a negative result: DCS matches baseline accuracy (76.8%) but uses 19.6% more tokens than standard self-consistency and 109% more than confidence-guided early stopping. The key insight is that approximately 96% of drafts complete within the token limit, causing the continuation mechanism to rarely activate meaningfully. This demonstrates that simpler early-stopping methods dominate two-stage branch budgeting when draft lengths are sufficient for most problems. Future work could exp[illegible] benchmarks where solutions genuinely require more tokens than the draft budget allows.

Index ID: #P05-A03
Full text moved to continuation pages.

Annotation Continuation · Source Page 5

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P05-A01 · SUGGESTION · MAJOR · #R015 · Part 1

This discussion and limitation framing is directionally right: if T_{draft} is long enough for most problems, continuation cannot add value. However, the current text still reads as a generalized lesson (“two-stage approaches may require…”) without performing the minimal confirming experiment that directly tests the proposed mechanism boundary.

To turn this into a strong negative-result paper, you should add one targeted experiment that *forces* the continuation mechanism to matter:
· Sweep T_{draft} downward while keeping total expected tokens comparable, and report how (i) draft completion rate, (ii) continuation activation rate, and (iii) accuracy/tokens change.
· Alternatively (or additionally), evaluate on a harder setting where solutions exceed 1024 more often.

Also, consider tying this to a broader budget-aware evaluation perspective: complex test-time strategies often appear beneficial only when they effectively spend more compute; your paper can explicitly argue that DCS does not create value per token under the tested regime.

Mentor Revised Version:
These results indicate that DCS is bottlenecked by a regime mismatch: with $T_{draft}{=}1024$, most MATH-500 drafts already terminate with a parseable answer, so continuation rarely changes the candidate set and mainly adds overhead. To validate this mechanism boundary, we will add a T_{draft} sweep (e.g., 128–1024) under matched total token caps and report draft-completion and continuation-activation rates alongside accuracy–cost curves. We will then bound our takeaway to the tested regime: continuation-based branch budgeting is unlikely to improve efficiency when the draft budget is comparable to typical solution length, but it may be relevant when drafts are deliberately short or tasks require longer solutions.

References:
[1] Reasoning in Token Economies: Budget-Aware Evaluation of LLM Reasoning Strategies 2406.06461

#P05-A02 · SUGGESTION · MAJOR · #R016 · Part 1

The related-work taxonomy is reasonable, but it does not clearly position DCS against the most similar family: methods that *reuse partial traces/prefixes* to guide subsequent sampling or to reweight votes. For instance, prefix-guided self-consistency variants can be very close in spirit (using early signals to steer later generation) and should be discussed explicitly, otherwise reviewers may see DCS as an unacknowledged re-derivation.

Concrete fix:
· Add one paragraph under “Self-Consistency and Variants” titled “Prefix-guided / partial-trace reuse” that contrasts:
(i) prefix-guided generation (guides new branches with a selected prefix),
(ii) thought-level pruning (removes redundant chains online),
(iii) DCS continuation (extends existing drafts selected by interim answers).
· For each, state the key difference in *what is reused* (prefix tokens vs interim answer vs embedding similarity) and *how compute is allocated* (generate new vs continue old vs stop).

Also, verify citation identifiers: e.g., the paper cites “Path-Consistency (Zhu et al., 2024)” but the actual arXiv ID (from retrieval) is 2409.01281. Use consistent bibliographic entries so readers can locate the exact work.

Mentor Revised Version:
Prefix-guided variants of self-consistency reuse partial traces to steer later computation. For example, path-consistency with prefix enhancement selects a high-confidence prefix from early samples and conditions subsequent sampling on that prefix, reducing redundant exploration [1]. Slim-SC prunes redundant chains online using thought-level similarity [2]. In contrast, DCS does not guide *new* sampling from a shared prefix; it allocates additional tokens to a small subset of already-generated drafts selected by an interim-answer vote histogram. This distinction clarifies that DCS is a continuation-based budget reallocation scheme rather than a prefix-guided resampling method.

References:
[1] Path-Consistency with Prefix Enhancement for Efficient Inference in LLMs 2409.01281

[2] Slim-SC: Thought Pruning for Efficient Scaling with Self-Consistency 2509.13990

#P05-A03 · SUGGESTION · MAJOR · #R017 · Part 1 · CONT.

The conclusion is clear, but it repeats the broad claim “simpler early-stopping methods dominate two-stage branch budgeting when draft lengths are sufficient” without (i) matched-budget frontier curves or (ii) sensitivity experiments that validate the boundary condition.

Annotation Continuation · Source Page 5 (cont. 2)

Continuation sheet linked to source highlights. Click marker headers to jump back to the paper region.

#P05-A03 · SUGGESTION · MAJOR · #R017 · Part 2

Two concrete improvements:
1) Bound the claim to the tested regime (MATH-500 + Qwen2.5-Math-7B-Instruct + $T_{draft}{=}1024$) and explicitly state what remains unknown.
2) End with a falsifiable checklist for future work, e.g., “DCS is worth trying when draft completion < X%,” “when solutions typically exceed T_{draft},” etc. This is a strong way to turn a negative result into actionable insight.

Mentor Revised Version:
In the evaluated setting (MATH-500 with Qwen2.5-Math-7B-Instruct at $T_{draft}{=}1024$ and $T_{cont}{=}1024$), DCS matches self-consistency accuracy but increases token cost, and confidence-guided early stopping achieves the same accuracy with fewer tokens. Diagnostic analysis indicates that the continuation mechanism is rarely exercised because most drafts already terminate with a parseable answer under the draft budget. Future work should test DCS in regimes where drafts are intentionally short or tasks are longer-horizon, and we expect continuation-based budgeting to be most relevant when (i) draft completion rate is low, (ii) interim-answer votes are informative early, and (iii) additional continuation tokens meaningfully change final answers under a fixed total budget.

REFERENCES

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. pp. 12375–12396, 2023.

Ehsan Aghazadeh, Ahmad Ghasemi, Hedyeh Beyhaghi, and Hossein Pishro-Nik. Cges: Confidence-guided early stopping for efficient and accurate self-consistency. *ArXiv*, abs/2511.02603, 2025.

Maciej Besta, Nils Blach, Aleš Kubíček, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, H. Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. pp. 17682–17690, 2023.

A. Feng, Marius Alonso, and Ambroise Odonnat. Optimal self-consistency for efficient reasoning with large language models. *ArXiv*, abs/2511.12309, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *ArXiv*, abs/2103.03874, 2021.

Colin Hong, Xu Guo, Anand Chaanan Singh, Esha Choukse, and Dmitrii Ustiugov. Slim-sc: Thought pruning for efficient scaling with self-consistency. *ArXiv*, abs/2509.13990, 2025.

Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. *ArXiv*, abs/2401.10480, 2024.

Rohin Manvi, Anikait Singh, and Stefano Ermon. Adaptive inference-time compute: Llms can predict if they can do better, even mid-generation. *ArXiv*, abs/2410.02725, 2024.

Guangya Wan, Yuqi Wu, Jie Chen, and Sheng Li. Reasoning aware self-consistency: Leveraging reasoning paths for efficient llm sampling. pp. 3613–3635, 2024.

Xuezhi Wang, Jason Wei, D. Schuurmans, Quoc Le, Ed H. Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *ArXiv*, abs/2203.11171, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, F. Xia, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *ArXiv*, abs/2201.11903, 2022.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *ArXiv*, abs/2409.12122, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, T. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *ArXiv*, abs/2305.10601, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *ArXiv*, abs/2310.04406, 2023.

Jiace Zhu, Yingtao Shen, Jie Zhao, and An Zou. Path-consistency with prefix enhancement for efficient inference in llms. 2024.

King Zhu, Hanhao Li, Siwei Wu, Tianshun Xing, Dehua Ma, Xiangru Tang, Minghao Liu, Jian Yang, Jiaheng Liu, Y. Jiang, Changwang Zhang, Chenghua Lin, Jun Wang, Ge Zhang, and Wangchunshu Zhou. Scaling test-time compute for llm agents. *ArXiv*, abs/2506.12928, 2025.